

C Programming Bugging Tools

Section 1: Error Fundamentals & Definitions

Core Terminology

- **Error:** An unintentional mistake or oversight introduced into source code by a developer (e.g., using an addition operator + instead of multiplication *).
- **Bug:** An undesired operational defect or incorrect result generated by a computer program during runtime due to an underlying error.
- **Debugging:** The structured process of discovering, isolating, and resolving defects within a software program.

Section 2: Error Categorization in C

Error Class	Overview	Root Cause	Code Illustration
Syntax Errors	Code breaks the structural rules of the C language grammar.	Omitted semicolons, mismatched braces, or misspelled keywords.	<code>int a=10 (missing ;)</code>
Compilation / Type Errors	Operations requested on	Assigning strings to integer	<code>int *p = "hello";</code>

Error Class	Overview	Root Cause	Code Illustration
	incompatible types or uninitialized variables.	pointers, referencing undeclared items.	
Linker Errors	Compiling succeeds, but object files fail to bind into a final executable.	Declaring functions without bodies, missing standard library references.	undefined reference to 'main'
Runtime Errors	The application crashes or halts unexpectedly mid-execution.	Zero-division operations or invalid memory access attempts.	int x=10/0;
Logical Errors	The program completes without crashing, but yields	Bad mathematical logic or improper loop	for(i=0;i<5;i++) instead of <=5

Error Class	Overview	Root Cause	Code Illustration
	flawed values.	termination conditions.	

Why Valid Syntax Can Yield Inaccurate Results

Compilers strictly evaluate whether code complies with C syntax and type conventions before outputting binary instructions. They cannot decipher human intentions.

- If a programmer writes `sum=a-b;` instead of `sum=a+b;`, the syntax remains completely legitimate. The compiler builds the program without complaint, but executing it produces wrong outputs due to flawed logic.

Scenario 1: Handling Syntax & Compilation Faults

When invalid code is compiled, diagnostic alerts output the specific file location, line number, column index, and error details.

Source File (sample1.c)

```
#include <stdio.h>

int main()
{
    int a=10
    printf("%d\n", a);
    return 0;
}
```

```
(kali@kali)-[~]
└─$ cd c_debugging_lab

(kali@kali)-[~/c_debugging_lab]
└─$ nano sample1.c

(kali@kali)-[~/c_debugging_lab]
└─$ cc sample.c
cc1: fatal error: sample.c: No such file or directory
compilation terminated.

(kali@kali)-[~/c_debugging_lab]
└─$ cc sample.c
sample1.c: In function 'main':
sample1.c:5:5: error: expected ',' or ';' before 'printf'
   5 |     printf("%d\n", a);
     |           ^

```

Scenario 2: Utilizing Compiler Warnings (-Wall, -Wextra)

Warnings call out code that is syntactically legal yet potentially risky. Warnings do not interrupt compilation, but ignoring them regularly introduces runtime bugs.

Source File (sample2.c)

```
#include <stdio.h>
```

```
int main()
```

 $\{$

```
int a;
```

```
printf("Hello\n");
```

```
return 0;
```

}

- -Wall: Activates a wide array of useful compiler warnings.
- -Wextra: Turns on supplementary warnings not covered by -Wall.

```
(kali@kali)-[~]
└─$ cd c_debugging_lab
(kali@kali)-[~/c_debugging_lab]
└─$ nano sample2.c
(kali@kali)-[~/c_debugging_lab]
└─$ cc -Wall -Wextra sample2.c
sample2.c: In function 'main':
sample2.c:4:9: warning: unused variable 'a' [-Wunused-variable]
    4 |     int a;
      |
(kali@kali)-[~/c_debugging_lab]
└─$
```

Scenario 3: printf-Based State Tracing (Logical Bugs)

Prior to launching interactive debuggers, tracking variables and program flow can be accomplished using targeted print outputs.

Variable Value Inspection

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

Practical Demonstration: Tracing Loop Behavior (sample3.c)

The program below sums values from 1 up to 4 instead of 1 to 5 because of a logic bug. Embedded debug print statements highlight the exact iteration behavior.

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i, sum=0;
```

```
    for(i=0;i<5;i++)
```

```
    {
```

```
        printf("DEBUG: entering iteration i=%d\n", i);
```

```
        sum=sum+i;
```

```
        printf("DEBUG: sum updated to = %d\n", sum);
```

```

}

printf("Final Sum=%d\n", sum);

return 0;
}

```

```

(kali@kali)~/c_debugging_lab$ gcc sample3.c -g -o sample3
(kali@kali)~/c_debugging_lab$ gdb ./sample3
(gdb) run
DEBUG: entering iteration i=0
DEBUG: sum updated to = 0
DEBUG: entering iteration i=1
DEBUG: sum updated to = 1
DEBUG: entering iteration i=2
DEBUG: sum updated to = 3
DEBUG: entering iteration i=3
DEBUG: sum updated to = 6
DEBUG: entering iteration i=4
DEBUG: sum updated to = 10
Final Sum=10
(gdb)

```

Section 3: Overview of GDB (GNU Debugger)

Background: What are GNU & GDB?

- **GNU:** A recursive acronym for "GNU's Not Unix!". It represents a free software operating system initiative launched by Richard Stallman in 1983 to deliver an open Unix-like environment.
- **GDB (GNU Debugger):** The core debugger utility of the GNU system, engineered to inspect variables, pause execution with breakpoints, and diagnose software crashes.

Reference Table: Essential GDB Commands

Command Format	Abbreviation	Purpose & Behavior
break <line / func>	b	Pauses program execution at a given line number or function.
run	r	Initiates application execution from the beginning.
next	n	Advances to the next line of code (steps over functions).
step	s	Advances to the next line of code (steps into functions).
print <var>	p	Displays the current value of a target variable/expression.
display <var>	disp	Prints variable values automatically after every single step.
continue	c	Resumes execution until reaching another breakpoint or exit.

Command Format	Abbreviation	Purpose & Behavior
list	l	Prints surrounding source code lines around current location.
info locals	info loc	Displays all local scope variables and current values.
backtrace	bt	Shows the active call stack (sequence of function calls).
info breakpoints	i b	Lists all active breakpoints alongside assigned IDs.
delete <id>	d	Removes a specific breakpoint (or all if omitted).
disable <id>	dis	Temporarily deactivates a breakpoint without removal.
enable <id>	en	Re-activates a previously disabled breakpoint.
quit	q	Terminates the current GDB debugging session.

Scenario 4: Diagnosing a Logic Fault via GDB

Source File with Logic Fault (sample4.c)


```
#include <stdio.h>

int main()
{
    int i, sum=0;
    for(i=1;i<5;i++)
    {
        sum=sum+i;
    }

    printf("Sum = %d\n", sum);
    return 0;
}
```

- **Expected Result:** 15 (1 + 2 + 3 + 4 + 5)
- **Actual Result:** 10 (1 + 2 + 3 + 4)

```
Applications
kali@kali: ~/c_debugging_lab
$ cd c_debugging_lab
$ nano sample4.c
$ gcc -g sample4.c -o sample4
$ gdb ./sample4
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample4...
(gdb) break 7
Breakpoint 1 at 0x1151: file sample4.c, line 7.
(gdb) run
Starting program: /home/kali/c_debugging_lab/sample4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample4.c:7
7       sum sum i;
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7       sum sum i;
1: i = 2
2: sum = 1
(gdb)
```

- **Analysis:** The loop halts when i reaches 5 without running the loop body for 5. The boundary condition must be changed from $i < 5$ to $i \leq 5$.

Scenario 5: Stepping Through Functions (next vs. step)

- **next:** Executes the line. If it contains a function call, the function executes entirely in the background without entering its body.
- **step:** Steps inside the function call, transferring control into the function body for line-by-line inspection.

Source File (sample5.c)

```
#include <stdio.h>
```

```
int square(int x)
```

```
{
```

```
    int result1;
```

```
    result1 = x * x;
```

```
    return result1;
```

```
}
```

```
int main()
```

```
{
```

```
    int result, n = 5;
```

```
    result = square(n);
```

```
    printf("Square = %d\n", result);
```

```
    return 0;
```

}

```
(kali@kali)~/c_debugging_lab
$ nano sample5.c

(kali@kali)~/c_debugging_lab
$ cc -g sample5.c -o sample5
gdb ./sample5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/?>
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample5...
(gdb) break main
Breakpoint 1 at 0x1156: file sample5.c, line 11.
(gdb) run
Starting program: /home/kali/c_debugging_lab/sample5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:11
11      int result, n = 5;
(gdb) next
12      result = square(n);
(gdb) step
square (x=5) at sample5.c:5
5          result1 = x * x;
(gdb) |
```

Scenario 6: Tracking Runtime Crashes with Backtrace (backtrace / bt)

When a program crashes abruptly (e.g., zero-division exceptions), GDB pinpoints the exact line where execution halted.

Source File (sample6.c)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10;
```

```
    int b = 0;
```

```
    int c = a / b;
```

```
    printf("Result = %d\n", c);
```

```
    return 0;
```

```
}
```

```
(kali@kali)-[~/c_debugging_lab]
└─$ nano sample6.c

(kali@kali)-[~/c_debugging_lab]
└─$ cc sample6.c -o sample6
./sample6
zsh: floating point exception ./sample6

(kali@kali)-[~/c_debugging_lab]
└─$
```

Scenario 7: Diagnosing Array Index Out-of-Bounds Errors

In C, memory bounds for arrays are not checked automatically at runtime. Accessing elements past declared boundaries triggers unpredictable behavior or segmentation faults.

Source File (sample7.c)

C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[5] = {1, 2, 3, 4, 5};
```

```
    printf("%d\n", arr[1000000]);
```

```
    return 0;
```

```
}
```

```
(kali@kali)~/c_debugging_lab
$ nano sample7.c

(kali@kali)~/c_debugging_lab
$ cc sample7.c -o sample7
./sample7
zsh: segmentation fault ./sample7

(kali@kali)~/c_debugging_lab
$
```

Scenario 8: Resolving Memory Segmentation Faults (SIGSEGV)

Segmentation Faults occur when an application attempts to access restricted memory locations (such as dereferencing NULL pointers).

Source File (sample8.c)

```
#include <stdio.h>

int main()
{
    int *p = NULL;

    *p = 10;

    printf("%d\n", *p);

    return 0;
}
```

Corrected Version (sample8_fixed.c)

```
#include <stdio.h>

int main()
{
```

```
int arr[5] = {1, 2, 3, 4, 5};  
  
printf("Element at index 4: %d\n", arr[4]);  
  
return 0;  
  
}
```



```
(kali@kali)~/c/debugging_lab  
$ nano sample0_fixed.c  
  
(kali@kali)~/c/debugging_lab  
$ cc sample0_fixed.c -o sample0_fixed  
./sample0_fixed  
Element at index 4: 5  
  
(kali@kali)~/c/debugging_lab  
$
```

Section 4: Systematic 8-Step Debugging Workflow

1. Identify and document expected vs. actual behavior.
2. Establish reliable steps to reproduce the bug.
3. Recompile code with diagnostic flags (-Wall -Wextra -g).
4. Narrow down suspicious code segments.
5. Inspect program state using printf statements or GDB breakpoints.
6. Apply a focused, minimal code modification.
7. Recompile and re-run test scenarios.
8. Validate the program against edge cases and boundary inputs.